

Pipeline con datos particionados

El *dataset* a menudo lo integran **múltiples archivos almacenados en particiones** del sistema de ficheros en el dispositivo de almacenamiento. [PyArrow](#) es la librería *Python* para la integración de la implementación *C++* de [Apache Arrow](#) que proporciona un formato de memoria columnar de alto rendimiento para un intercambio de datos rápido y sin copias (**Zero-Copy IO**), sólo buffers *Arrow*. A diferencia de los enfoques convencionales que requieren múltiples copias de datos durante las transferencias, *Arrow* mantiene los datos en su formato columnar a lo largo de todo el proceso. Esto significa que los datos se mueven entre actividades sin la sobrecarga de los ciclos de serialización/deserialización, lo que reduce drásticamente el uso de CPU y la demanda de memoria. Provee una integración perfecta con *pandas*, [NumPy](#) y varios formatos de archivo (*Parquet*, *CSV*). Este ejemplo introductorio invoca a la función [pyarrow.dataset.dataset\(\)](#) para leer todos los archivos y presentarlos en un resultado único



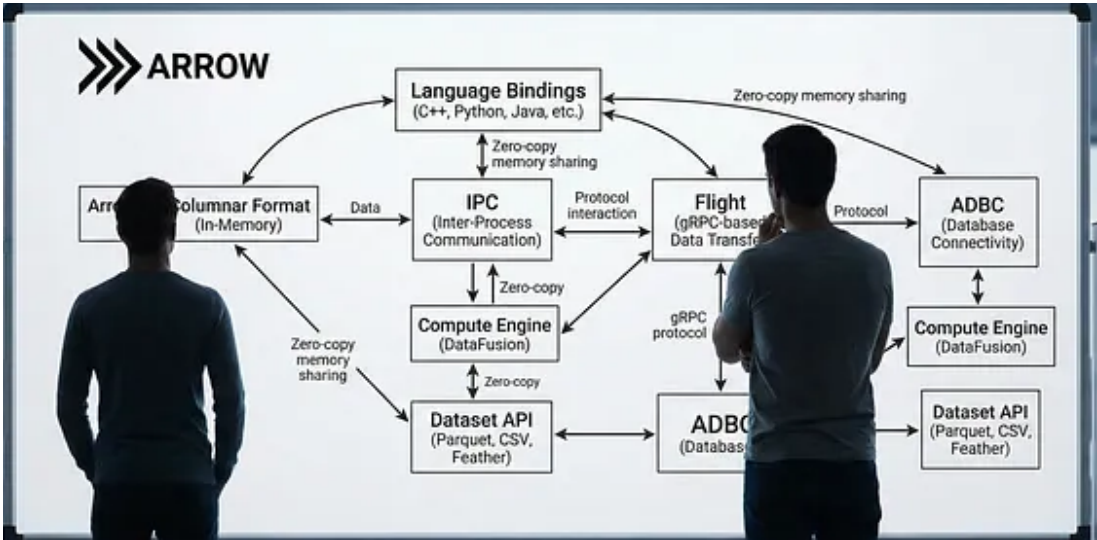
NYC Taxi Dataset

TLC Trip Record Data

Field Name	Description
VendorID	A code indicating the TPEP provider that provided the record. 1= Creative Mobile Technologies, LLC; 2= VeriFone Inc.
tpcp_pickup_datetime	The date and time when the meter was engaged.
tpcp_dropoff_datetime	The date and time when the meter was disengaged.
Passenger_count	The number of passengers in the vehicle. This is a driver-entered value.
Trip_distance	The elapsed trip distance in miles reported by the taximeter.
PULocationID	TLC Taxi Zone in which the taximeter was engaged
DOLocationID	TLC Taxi Zone in which the taximeter was disengaged
RateCodeID	The final rate code in effect at the end of the trip. 1= Standard rate 2=JFK 3=Newark 4=Nassau or Westchester 5=Negotiated fare 6=Group ride
Store_and_fwd_flag	This flag indicates whether the trip record was held in vehicle memory before sending to the vendor, aka "store and forward," because the vehicle did not have a connection to the server. Y= store and forward trip N= not a store and forward trip
Payment_type	A numeric code signifying how the passenger paid for the trip. 1= Credit card 2= Cash 3= No charge 4= Dispute 5= Unknown 6= Voided trip
Fare_amount	The time-and-distance fare calculated by the meter.
Extra	Miscellaneous extras and surcharges. Currently, this only includes the \$0.50 and \$1 rush hour and overnight charges.
MTA_tax	\$0.50 MTA tax that is automatically triggered based on the metered rate in use.
Improvement_surcharge	\$0.30 improvement surcharge assessed trips at the flag drop. The improvement surcharge began being levied in 2015.
Tip_amount	Tip amount – This field is automatically populated for credit card tips. Cash tips are not included.
Tolls_amount	Total amount of all tolls paid in trip.
Total_amount	The total amount charged to passengers. Does not include cash tips.

1. Scan *Parquet* como tabla *Arrow*

Documentación oficial



El siguiente código muestra la [lectura de datos en formato Parquet particionado](#) utilizando un `pyarrow.dataset`

Muestreo Pandas (Opcional)

```
# Filtrar datos en Pandas Dataframe
filtro = nyc_df[(nyc_df.total_amount > 100) & (nyc_df.w == "w=17")]

# Seleccionar columnas (proyección vertical)
res = filtro[["d", "total_amount", "passenger_count"]]

res.head(25)
```

2. Consulta *DuckDB* directa a los datos en la tabla *Arrow*, sin copias

La integración de *Arrow* en *DuckDB* permite consultar los datos en la tabla directamente, sin intermediarios 🤖🤖

```
con = duckdb.connect()
con.register("arrow_t", nyc_table) # zero-copy registration
```

```
res = con.execute("""
    SELECT w, d, VendorID, tpep_pickup_datetime, tpep_dropoff_datetime,
           passenger_count, trip_distance, total_amount
    FROM arrow_t
    WHERE (total_amount > 100) AND (w LIKE 'w=17')
    LIMIT 10
""").fetch_arrow_table() # retorno Arrow, sigue zero-copy

# Print res como DuckDB relation
con.from_arrow(res).show

# Siempre, siempre, siempre cerrar las conexiones
con.close()
```